

Agent memory in depth

LESSON 05 · KNOWLEDGE & RAG

What you'll learn

- Turn memory on for an agent in Studio and read its two settings without guessing what they do
- Tell a session apart from a user, and decide which one your memory should be scoped to
- Configure cross-session memory so a returning customer is recognised instead of starting over
- Prove memory works by storing a fact in one chat and recalling it from a brand-new one
- Connect Studio's memory scope to how you pass session IDs when you call the agent's API

Key concepts

Memory is what stops every conversation from starting at a blank slate. Without it, a customer has to reintroduce themselves on every visit; with it, the agent remembers the person and the things they told it. The interesting work isn't switching memory on — it's deciding what counts as the same conversation and what counts as the same person.

Memory is a feature you toggle, and the default provider is the right answer for almost everyone. You enable it under Features, which opens its settings; the first choice is the provider. Studio gives you a few, but the production-grade default (Cognee) is one you should leave untouched unless you have a specific reason not to.

The setting that actually matters is scope, and it's the one people get wrong. Two ideas make it clear: a session is one conversation, and a user is one person who may have many conversations over time. By default memory lives inside a single session, so a brand-new chat starts clean. Turn cross-session on and memory follows the person across all their conversations.

Extraction instructions let you steer what the agent bothers to remember. There's a separate tab where you can shape what gets stored, but the default is fine for most agents — you don't need to touch it to get useful memory. Reach for it only when you want tighter control over what the agent retains.

Out in your own product, you control conversations through the session ID you pass to the API. On the Deploy tab the agent exposes an API; when you call it you pass the agent's ID and a session ID. Reusing the same session ID continues a conversation, while a new one starts fresh — and because cross-session ties memory to the person behind those sessions, you manage the sessions and Lyzr handles the memory.

TAKEAWAY

Memory is what stops every conversation from starting at a blank slate.

TAKEAWAY

Memory is a feature you toggle, and the default provider is the right answer for almost everyone.

TAKEAWAY

The setting that actually matters is scope, and it's the one people get wrong.

TAKEAWAY

Extraction instructions let you steer what the agent bothers to remember.

TAKEAWAY

Out in your own product, you control conversations through the session ID you pass to the API.

In Studio

Primary surface: **Connections → Memory** — toggling memory on, choosing the provider, and setting scope, then proving it in the Playground.

1. Open your agent and go to **Connections → Memory** → switch on **Memory**. Turning it on opens its settings.
2. Under **provider**, leave the default (**Cognee**) selected — it's production-grade and you don't need to change it.
3. Find the **cross-session** setting. Off (the default) means memory lives inside a single session and each new chat starts clean. Switch it **on** so memory follows the person across all their conversations.
4. Optionally open the **extraction instructions** tab to steer what the agent remembers. The default is fine, so leave it unless you have a reason to narrow it.
5. **Save** the settings and **update the agent** so the changes take effect.
6. Open the **Playground** and tell the agent something personal — for example your name and that you prefer email updates. It acknowledges and quietly stores it.
7. Start a **completely new chat** (a different session). Ask who you are and how you like to be contacted — with cross-session on, it still answers correctly. That's the proof: it remembered the person, not just the conversation.
8. Go to the **Deploy** tab to see the agent's **API**. When you call it you pass the agent's ID and a **session ID** — the same session ID continues a conversation, a new one starts fresh.

TRY THIS

- 1** On a test agent, switch on Memory under Features, keep the default provider, and leave cross-session **off**. In the Playground tell it your name, then open a new chat and ask who you are — confirm it has forgotten. Now turn cross-session **on**, save, and repeat the test to watch the behaviour flip.
- 2** Store a preference ("I prefer email updates") in one chat, then start a fresh chat and ask the agent how you like to be contacted. Confirm it recalls the preference across sessions, not just within the same conversation.
- 3** Open the Deploy tab and find where you pass the **session ID**. Write down, in one sentence each, what reusing the same session ID does versus passing a new one — and how cross-session memory changes a returning user's experience either way.